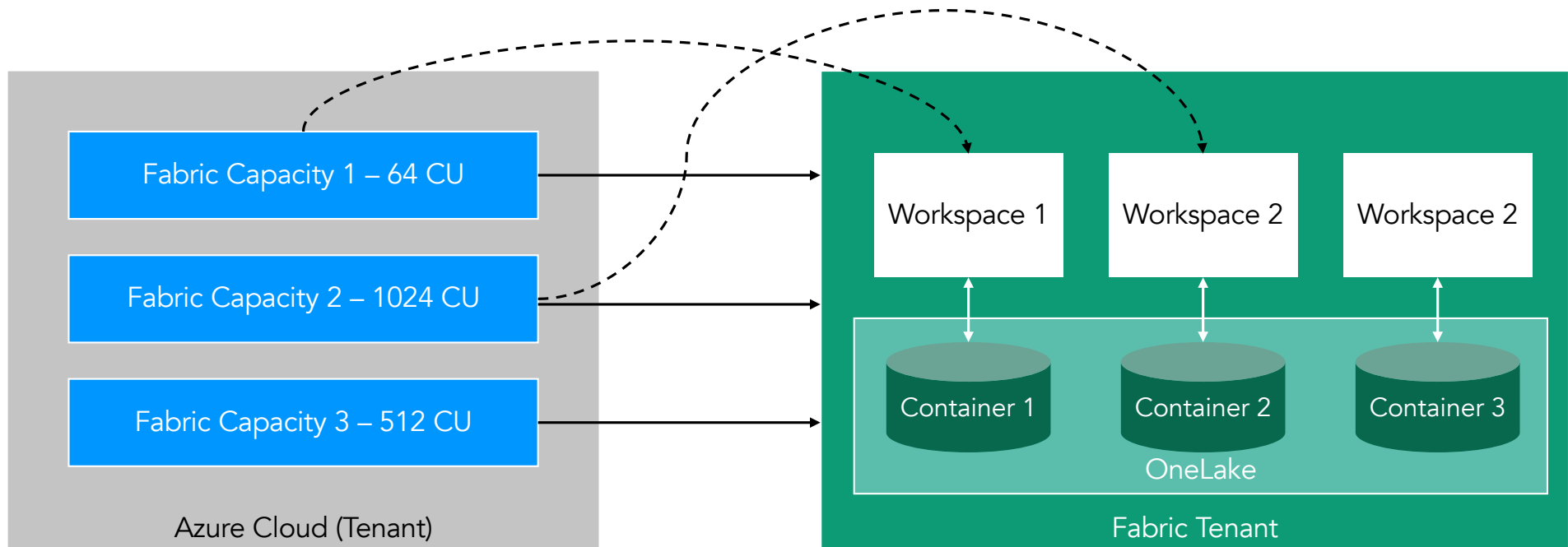


# Fabric Data Engineering

[www.3aayaam.in](http://www.3aayaam.in)

# Fabric & Azure



[www.3aayaam.in](http://www.3aayaam.in)

# Data Engineering Workload

- Purpose – Collect, Store, Process & Analyze.
- Data Engineering workload contains:
  - Lakehouse.
  - Spark Job Definition.
  - Spark Notebook.
  - Pipeline.

# Data Lakehouse (Delta Lake Tables)

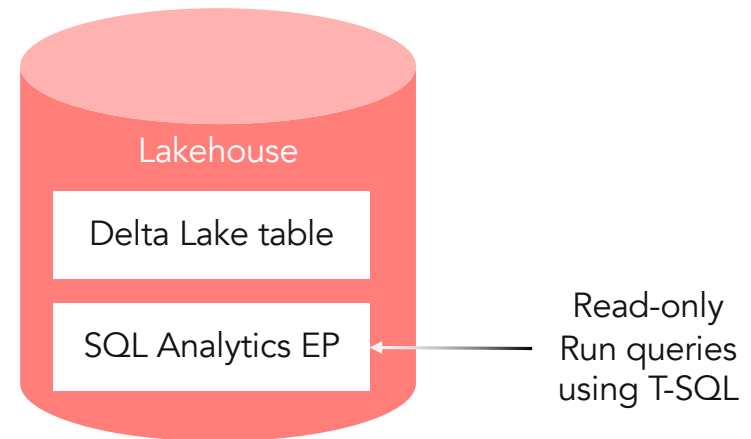
[www.3aayaam.in](http://www.3aayaam.in)

# Lakehouse

- Lakehouse = Data Lake + Data Warehouse.
- Data Lake – Object level operation.
- Data Warehouse – Row level operation using ACID transactions.
- Lakehouse provides scalability of data lake and row level ACID capability of warehouse.
- Lakehouse – Iceberg, Delta Lake, HUDI.
  - Metadata layer to implement row level modification using transactions.
- Fabric Lakehouse – Delta Lake.

# Fabric Lakehouse

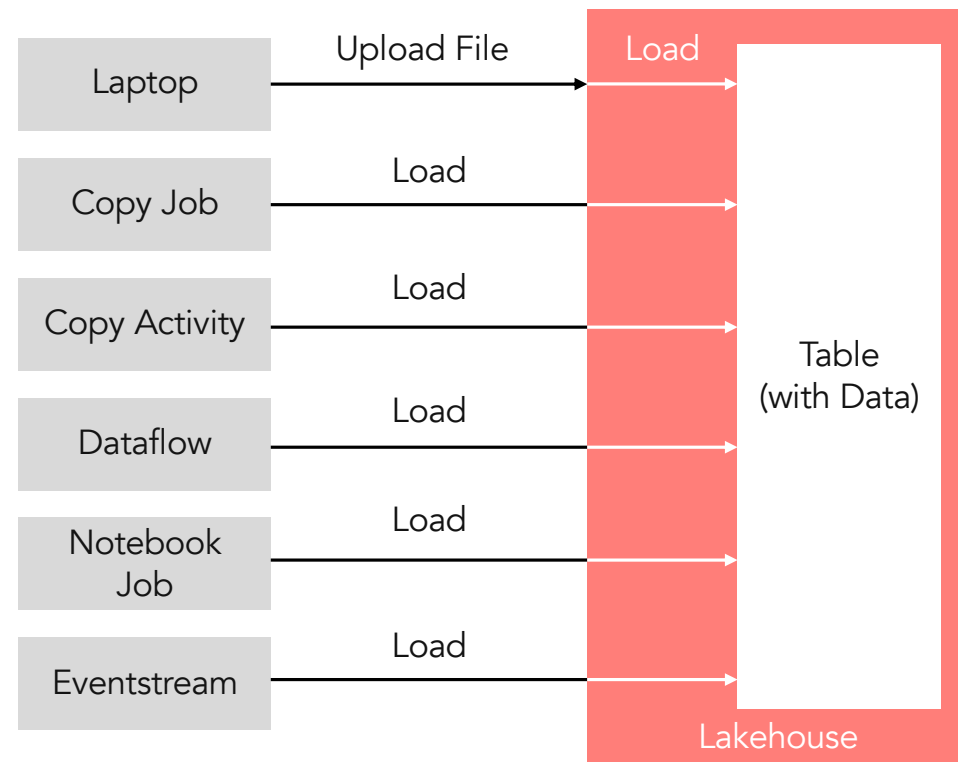
- Lakehouse Delta Parquet table.
- SQL Analytics Endpoint.
- Accessing Lakehouse tables from Notebook or Spark Job.
  - `workspace.lakehouse.schema.table`
  - Default schema – `dbo`.
- Materialized Lake View.
  - View that has its data stored physically in storage.
  - `CREATE MATERIALIZED LAKE VIEW <name>`.
- Refresh MV.
  - Optimal refresh – Automatically selects optimal strategy : Incremental, Full & No refresh.
  - Full refresh



| Refresh Policy      | Description                                                                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Incremental refresh | An incremental refresh only processes the changed data in the sources referenced in the materialized lake views definition.                               |
| No refresh          | If the source remains unchanged, i.e., if no change detected in delta commits, the service skips the refresh. This behavior saves unnecessary processing. |
| Full refresh        | A full refresh involves assessing the entire dataset of dependent sources whenever any modification is made to the source.                                |

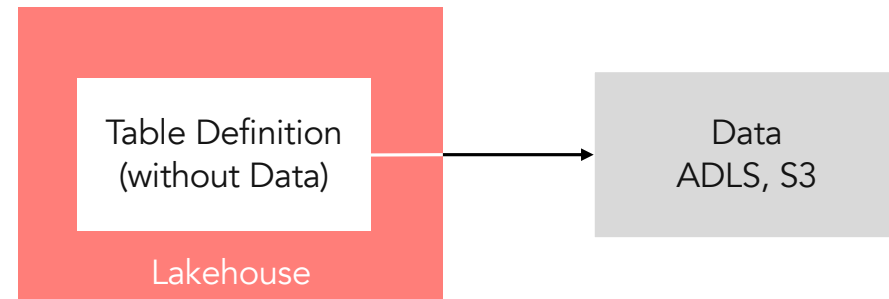
# Load Data into Lakehouse

- File load – CSV, Parquet, ORC.  
Copy job, Copy activity, Dataflow, Notebook, Spark job.
- Real-Time ingestion – From Eventhouse.



# Load Data into Lakehouse

- Shortcuts. (Hive External Tables).
  - Define metadata (schema) on lakehouse.
  - Data would be somewhere else. Other workspace, ADLS Gen2, AWS S3.





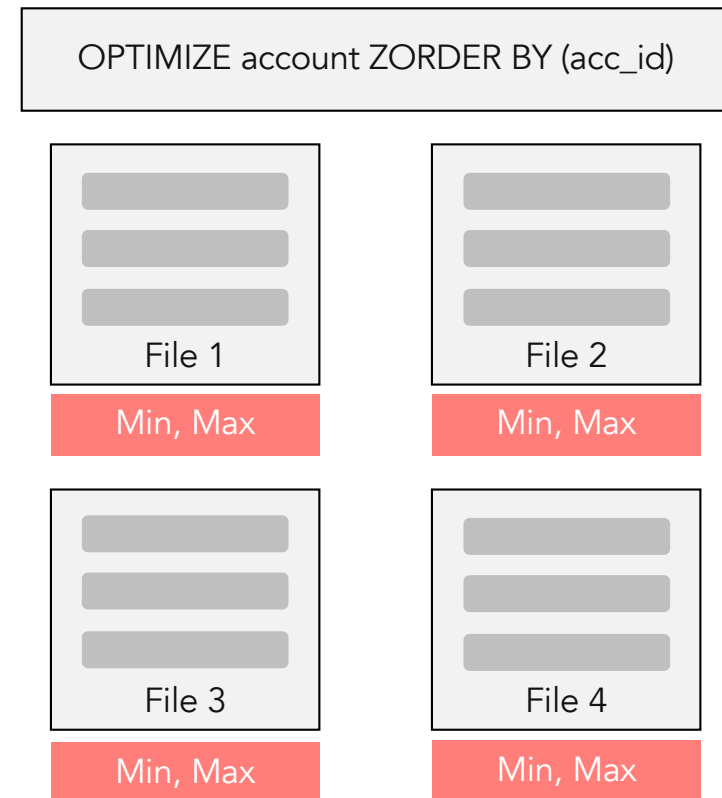
# Fabric V-Order

- Optimization in Parquet file format.
- Faster reads using Fabric Compute Engines (T-SQL, Spark, Power BI). Writes are impacted.
- Compatible with Parquet and Delta Tables.
- No changes required on any of the compute engines for reading and writing.
- All Delta table features are available as-is.
  - Time travel.
  - Vacuum.
  - Z-Order.
  - Compaction.

| Configuration                                                | Default Value      | Description                                                                                            |
|--------------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------|
| <code>spark.sql.parquet.vorder.default</code>                | <code>false</code> | Controls session-level V-Order writing. Set to <code>false</code> by default in new Fabric workspaces. |
| <code>TBLPROPERTIES("delta.parquet.vorder.enabled")</code>   | Unset              | Controls default V-Order behavior at the table level.                                                  |
| DataFrame writer option: <code>parquet.vorder.enabled</code> | Unset              | Used to control V-Order at the write operation level.                                                  |

# Delta Lake Z-Order

- Delta Table Feature.
- Sorts rows physically based on columns (that are mentioned while table optimization).
- Metadata storage of each files.
- Steps:
  - CREATE table
  - OPTIMIZE table ZORDER BY (col1, col2)
- Which columns should be put in Z-Order?
  - Highly cardinal columns, used in WHERE clause.



# Table Maintenance Operations

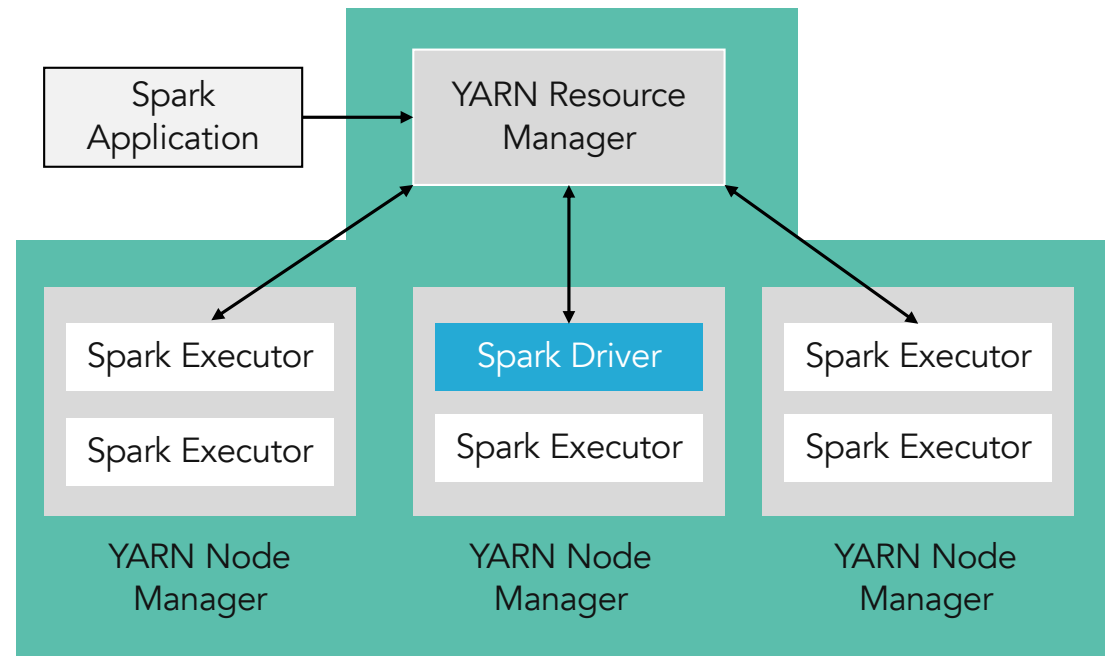
- Table maintenance operations:
  - Optimize command – Merges small Parquet files into a large file.
  - V-Ordering – Executes with the optimize command to compress, sort and encode Delta parquet files.
  - Vacuum command – Deletes old files that are not needed anymore.

# Spark on Fabric

[www.3aayaam.in](http://www.3aayaam.in)

# Spark on Fabric

- Apache Spark is world's most famous distributed data processing framework.
- Spark is used in batch ingestion, batch processing and stream processing of data.
- Spark runs on a cluster that is managed by cluster manager tools like YARN.
- Fabric is a hosting platform for Spark.



# (SPARK + KAFKA + More) Batch & Stream Processing - Data Engg. Vol 2



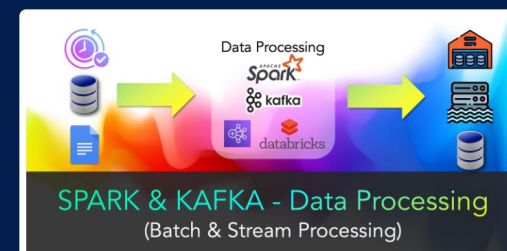
★★★★★ 5.0 (3 ratings)

Created by Soumyadeep Dey

 English

## About the course

This is **Volume 2 of Data Engineering** course. In this course I will talk about Open Source Data Processing technologies - **Spark and Kafka**, which are the most used and most popular data processing frameworks for **Batch & Stream Processing**. In this course you will learn **Spark from Level 100 to Level 400 with real-life hands on and projects**. You will get introduced to Data Lake on AWS (that is S3) & Data Lakehouse using **Apache Iceberg**.



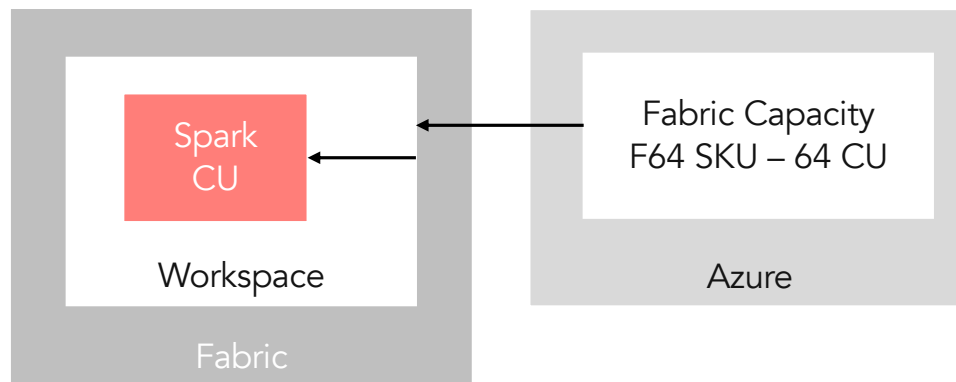
## (SPARK + KAFKA + More) Batch & Stream Processing - Data Engg. Vol 2

Buy now for ₹6,999

**Language:** English

**Instructors:** Soumyadeep Dey

# Spark on Fabric



| Scenario                                            | Typical Startup Time                                    |
|-----------------------------------------------------|---------------------------------------------------------|
| Default settings, no libraries                      | 5 – 10 seconds                                          |
| Default settings + library dependencies             | 5 – 10 seconds + 30 seconds – 5 min (for library setup) |
| High traffic in region, no libraries                | 2 – 5 minutes                                           |
| High traffic + library dependencies                 | 2 – 5 minutes + 30 seconds – 5 min (for libraries)      |
| Network security (Private Links/VNet), no libraries | 2 – 5 minutes                                           |
| Network security + library dependencies             | 2 – 5 minutes + 30 seconds – 5 min (for libraries)      |

- Spark clusters use capacity unit from Workspace.
- Starter Pool – Readily available Spark clusters. Spark session can be initialized within 5 to 10 seconds. Can take more time if ...
  - There are custom libraries and Spark configuration properties.
  - High traffic in the region.
  - Private Link or VNet is configured.

# Spark Custom Pool

- Spark Pool
  - No. of nodes – 1 Head node + Worker nodes.
  - Size of each node.
- Automatic deactivation. Default – 2 mins, configurable up to 20 mins.
- Spark pools are provisioned (started with nodes) when Notebook or Spark Job is run. 3 - 5 mins to start.
- 1 CU – 2 Spark VCores.
  - 2 tasks per CU.

| Size     | vCore | Memory |
|----------|-------|--------|
| Small    | 4     | 32 GB  |
| Medium   | 8     | 64 GB  |
| Large    | 16    | 128 GB |
| X-Large  | 32    | 256 GB |
| XX-Large | 64    | 512 GB |

- Autoscale – Scale no. of nodes up and down.
- Dynamic Allocation – Automatically scale no. of Spark executors up and down based on requirement.



# Spark Capacity Settings

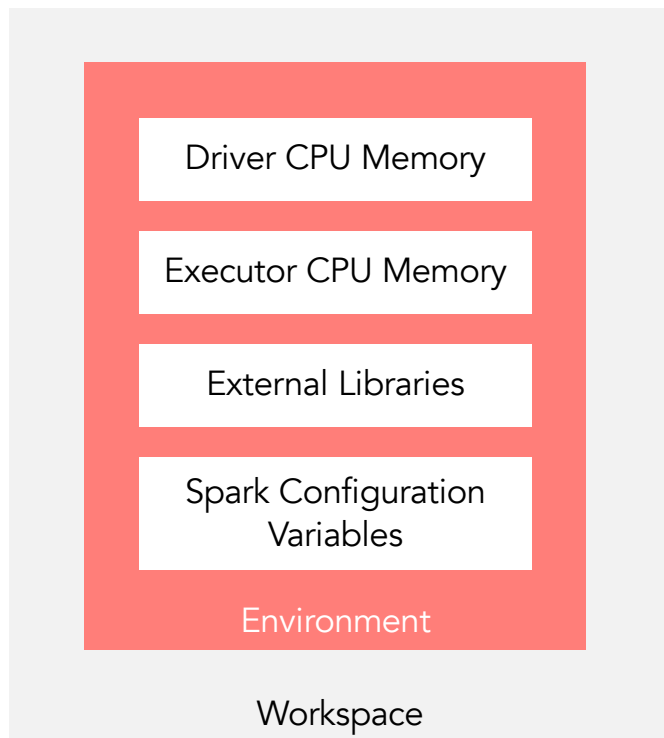
- Fabric Tenant level.
- Workspace level.

# Spark Capacity

| SKU name         | Capacity units | Spark VCores | Node size | Default max nodes | Max number of nodes |
|------------------|----------------|--------------|-----------|-------------------|---------------------|
| F2               | 2              | 4            | Medium    | 1                 | 1                   |
| F4               | 4              | 8            | Medium    | 1                 | 1                   |
| F8               | 8              | 16           | Medium    | 2                 | 2                   |
| F16              | 16             | 32           | Medium    | 3                 | 4                   |
| F32              | 32             | 64           | Medium    | 8                 | 8                   |
| F64              | 64             | 128          | Medium    | 10                | 16                  |
| (Trial Capacity) | 64             | 128          | Medium    | 10                | 16                  |
| F128             | 128            | 256          | Medium    | 10                | 32                  |
| F256             | 256            | 512          | Medium    | 10                | 64                  |
| F512             | 512            | 1024         | Medium    | 10                | 128                 |
| F1024            | 1024           | 2048         | Medium    | 10                | 200                 |
| F2048            | 2048           | 4096         | Medium    | 10                | 200                 |

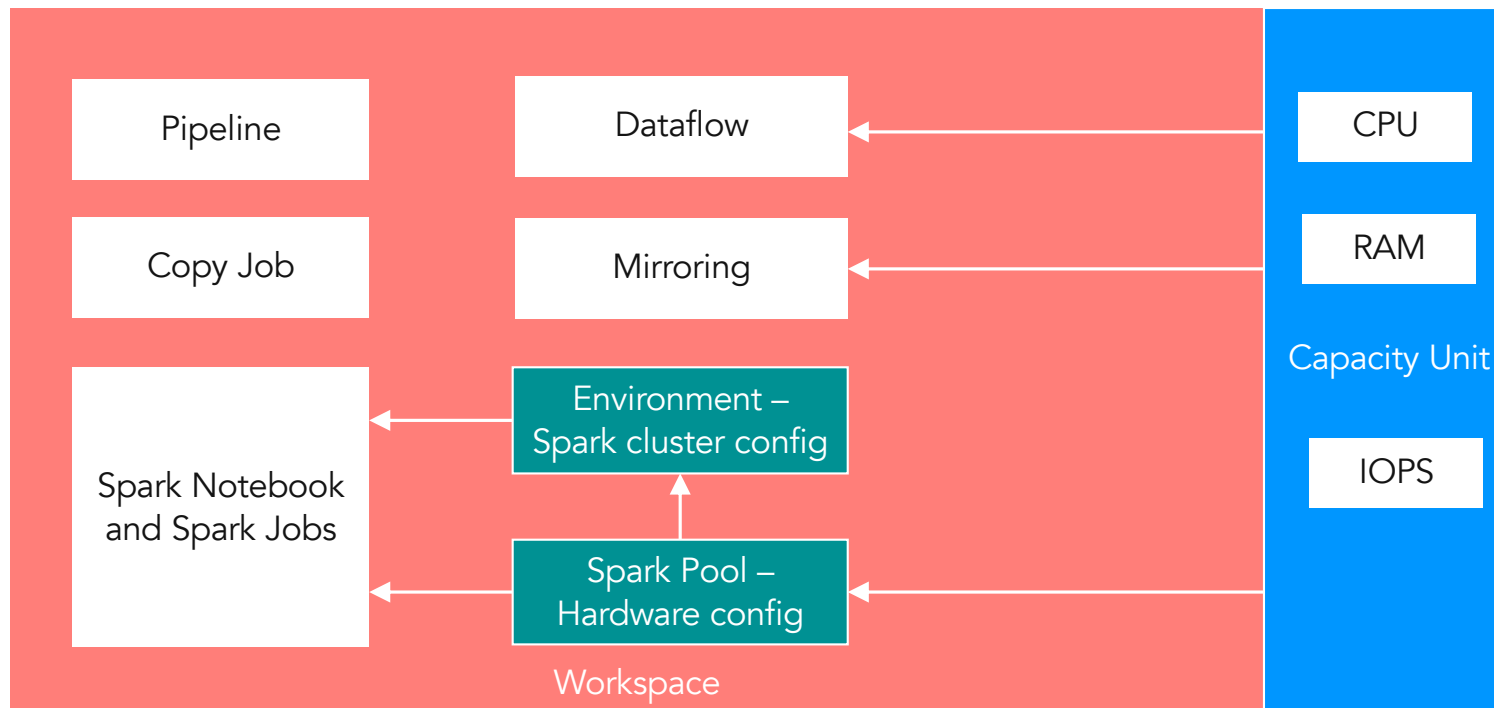
- Setup Starter Pool.
- Create a new Spark Pool.

# Environment



- Environment is created inside of a Workspace and associated with a workspace.
- Environment is used to configure ...
  - Compute resources for Spark execution.
  - Spark driver & executor configuration.
  - External libraries.
  - Spark configuration variables.
- Acceleration – Native engine from Fabric.
  - Optimized engine for Spark.
  - Velox (C++) & Apache Gluten.
  - Runs on Lakehouse infrastructure.
  - Claimed 4x performance improvement – TPC-DS 1TB.

# Workspace + Pool + Env



# Acceleration Benefits

- Columnar processing & Vectorization.
- Data must be in Parquet and Delta (Lakehouse) format.
- Queries with complex transformations and aggregations.
- Queries that are compute intensive and not IO bound.

# Microsoft Spark Utilities

- Built-in package to work on file systems, run & execute notebooks etc.

- `from notebookutils import mssparkutils`

- `mssparkutils.fs.ls()`
- `mssparkutils.fs.mkdirs()`
- `mssparkutils.fs.cp()`
- `mssparkutils.fs.fastcp()`
- `mssparkutils.fs.head()`
- `mssparkutils.fs.mv()`
- `mssparkutils.fs.put()`
- `mssparkutils.fs.append()`
- `mssparkutils.fs.rm()`
- `mssparkutils.notebook.run()`

# Fabric Notebook

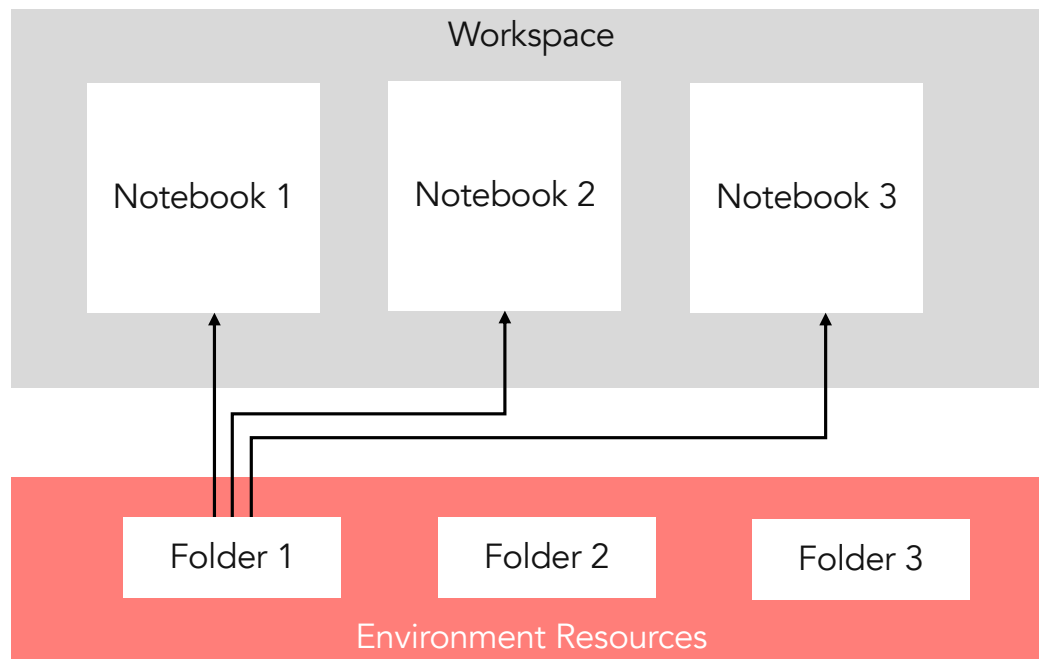
[www.3aayaam.in](http://www.3aayaam.in)

# Fabric (Spark) Notebook

- Interactive, web-based interface for Spark coding by Data Engineers, and Machine Learning codes by Data Scientists.
- Similar to Jupyter notebook.
- Should only be used for Spark Batch Processing.
  - Ingest, transform and load data.
- Execution on Spark code:
  - Interactive execution.
  - Notebooks can be used as Pipeline activities.
- Notebook code execution – i) Interactive. ii) Pipeline. iii) Scheduled.
- Let's explore Fabric Notebook.



# Environment & Notebook



- Should not be used for data files.
- Should be used to share config files, codes etc.
- Modification done to the files persist only for the session.

# Access ADLS Files from Notebook

- How to access ADLS Container from Spark Notebook.



```
spark.conf.set("fs.azure.account.auth.type.<storage_acc>.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type.<storage_acc>.dfs.core.windows.net", "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id.<storage_acc>.dfs.core.windows.net", <client_id>)
spark.conf.set("fs.azure.account.oauth2.client.secret.<storage_acc>.dfs.core.windows.net", <client_secret_value>)
spark.conf.set("fs.azure.account.oauth2.client.endpoint.<storage_acc>.dfs.core.windows.net", "https://login.microsoftonline.com/<tenant_id>/oauth2/token")
```

# NotebookUtils

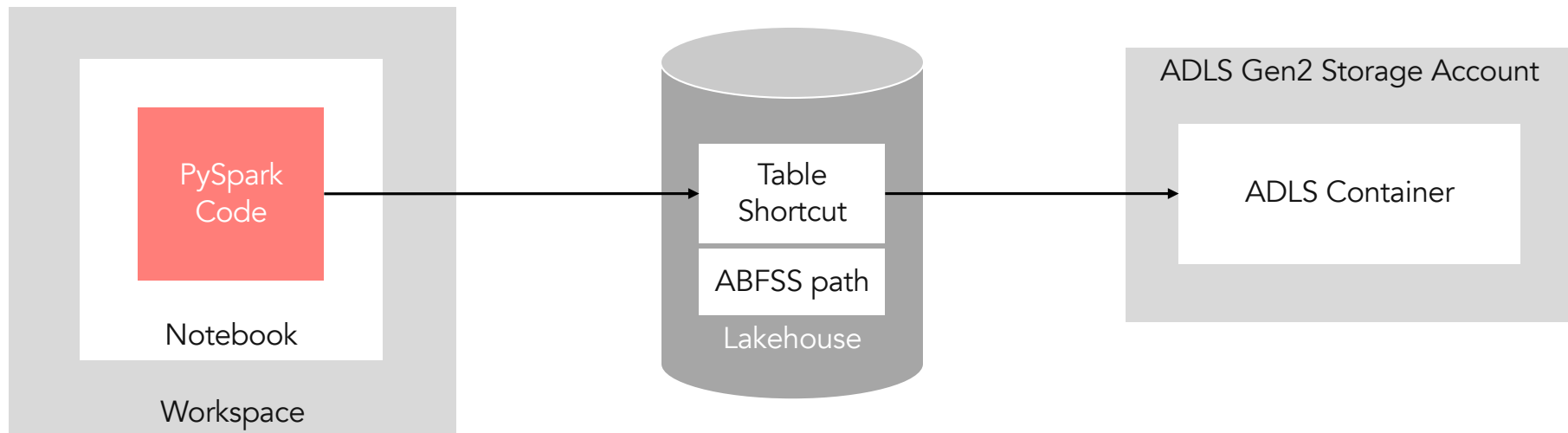
- Built-in package to perform various tasks in Fabric Notebook.
- <https://3aayaamfabricstorageacc.dfs.core.windows.net/3aayaam-fabric-adls-container>
- [abfs://3aayaam-fabric-adls-container@3aayaamfabricstorageacc.dfs.core.windows.net/<directory>](#)
- File System Commands:
  - `notebookutils.fs.ls('adls-dir-path')`
  - `notebookutils.fs.cp('source dir or file', 'destination dir or file')`
  - `notebookutils.fs.fastcp('source dir or file', 'destination dir or file')`
  - `notebookutils.fs.append('file path', 'content')`
  - `notebookutils.fs.put('file path', 'content')`
  - `notebookutils.notebook.run("notebook1", timeout, <parameterMap>, workspaceId)`
  - `notebookutils.notebook.runMultiple([ "notebook1", "notebook2" ])`

# NotebookUtils

- Credential Utilities – Used to get access tokens from Azure Key Vault.
  - `notebookutils.credentials.getToken('Key')`
  - `notebookutils.credentials.getSecret('https://<name>.vault.azure.net/', 'secret name')`
- ADLS file mounts – Used to mount ADLS storage to Spark driver and executor/worker nodes.
  - `secret_key = notebookutils.credentials.getSecret('Key')`
  - `notebookutils.fs.mount("abfss://...dfs...", "mount_dir", { "sasToken" : secret_key })`

# Access ADLS Files using Shortcut

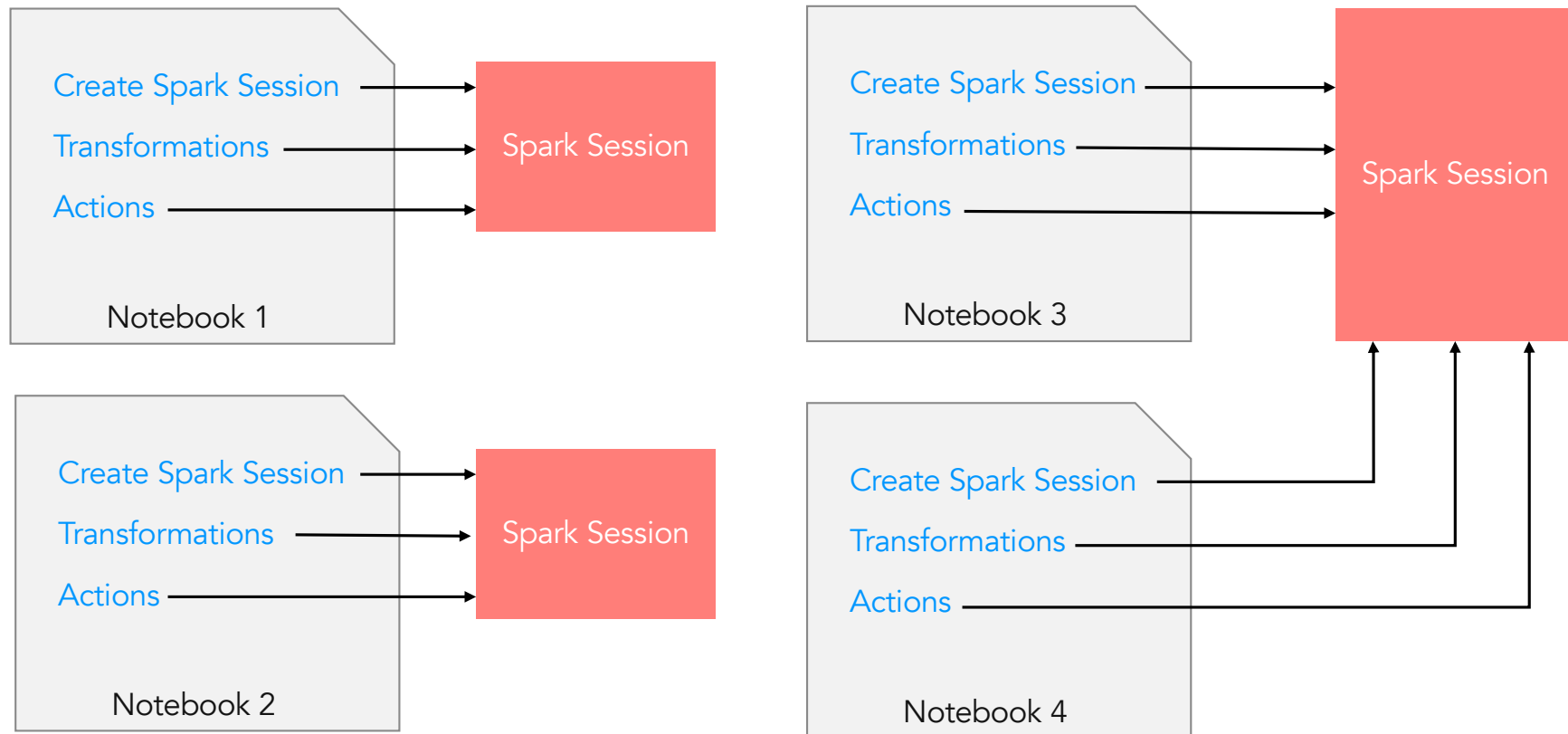
- Access ADLS using Lakehouse Shortcut from notebook fs utilities.



# Notebook Commands

- Notebook Utilities
  - `notebookutils.notebook.run("<notebook-name>", <timeout>, { "parm1": val1, "parm2": val2})`
  - `notebookutils.notebook.runMultiple([ "notebook-1", "notebook-2"])`
- Lakehouse Commands
  - `notebookutils.lakehouse.create("<name>")`
  - `notebookutils.lakehouse.delete("<name>")`
  - `notebookutils.lakehouse.loadTable(...)`

# High Concurrency



# High Concurrency

- All notebooks should be run by the same user.
- Should have the same Spark compute configurations.
- Should have same library packages.
- Can be used in Pipeline using Session Tags.

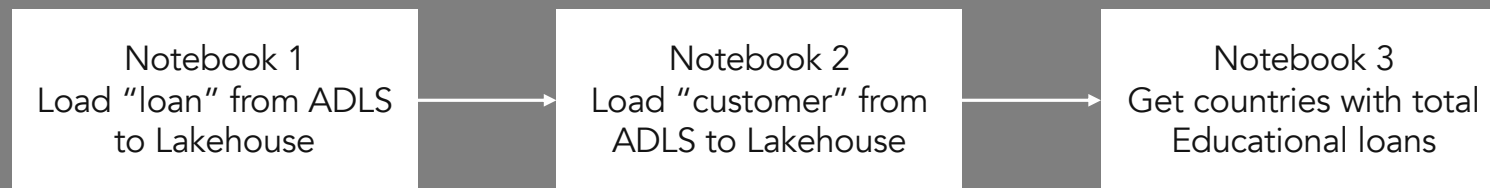


# PySpark on Notebook

- Set Spark conf variable for authentication.
- Read from ADLS Gen2.
  - `spark.read.format("csv").option().option().load("abfss://<path>")`
- Load/Write to Lakehouse.
  - Lakehouse Table – `df.write.format("delta").mode().saveAsTable("<table-name>")`
  - Path – `df.write.format("delta").mode().save("Tables/<table-name>")`
  - Files – `df.write.format("parquet").mode().save("Files/RawData/<file-name>")`
- Read from Lakehouse.
  - Lakehouse Table – `spark.read.table("<table-name>")`
  - Path – `spark.sql(<SQL Statement>)`
  - Files – `spark.read.format(parquet).load("Files/RawData/<file-name>")`

# HANDS-ON : ADLS, Transform, Lakehouse

- Notebook 1 - Read from ADLS (loan & customer) and load to Lakehouse.
- Notebook 2 – Find the country that has taken highest education loan.
- Run both the Notebooks one after the another in a Pipeline.



# T-SQL & Mirrored DB on Notebook

- T-SQL
  - Data Warehouse is needed as source.
  - SQL Analytics Endpoint for data warehouse
- Mirrored DB

# EXERCISE : Fabric Notebook

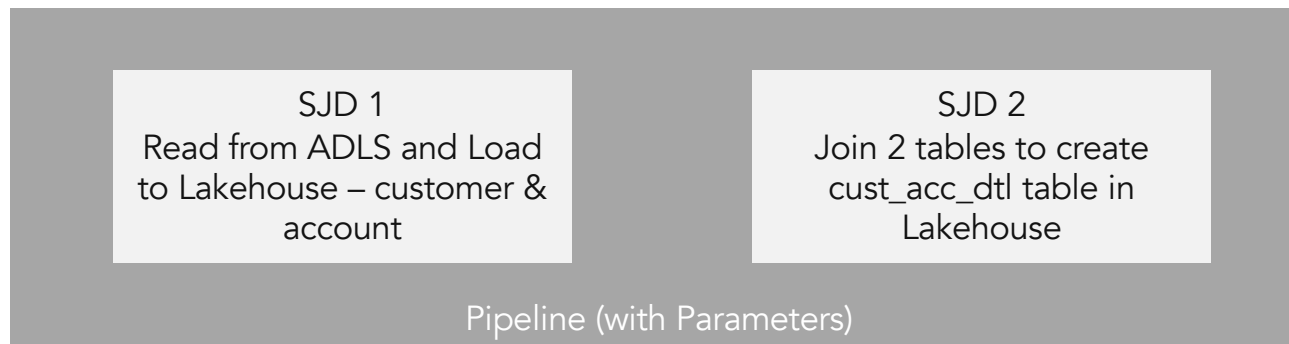
- Load all the following files in their corresponding Lakehouse tables. All table names should have \_lh at the end. Use ForEach pipeline activity.
- Run the following use cases. Output of all should be saved in their corresponding Lakehouse tables. Run all use cases one after the other in a pipeline.
  - List all cross-country transactions.
  - Find top 5 cross-country transactions by amount. (Sum of all transactions from source country to destination country).
  - Which are the top 10 countries that invest in stock market trading (ledger\_type = 'Equity') by volume. What is the corresponding invested amount for each countries.
  - Find total amount for each loan type for individual countries.

# Spark Job Definition

[www.3aayaam.in](http://www.3aayaam.in)

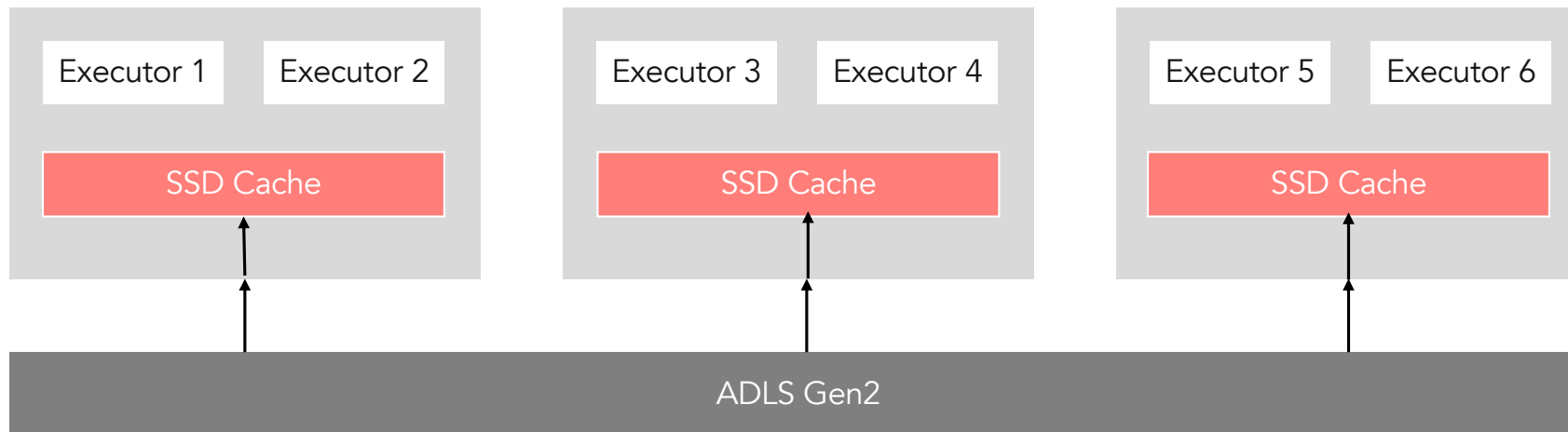
# Spark Job Definition (SJD)

- Run Spark jobs on cluster.
- Both batch & streaming jobs.
- Lakehouse must be associated with the Job Definition.
- Can be run using Pipeline.



# Intelligent Cache

- Caches files read from ADLS to local SSDs in Spark Cluster (pool).
- `spark.conf.set("spark.synapse.vegas.useCache", "false/true")`
- Each read request checks whether the file is in cache or is stale.
- Cache size cannot be configured. 50% of the available storage.



# EXERCISE : SJD

- Load all the files in their corresponding Lakehouse tables.
- Run the following use cases. Output of all should be saved in their corresponding Lakehouse tables. Run all use cases one after the other in a pipeline.
  - Which type of transaction (ledger\_type) dominates the most in cross-country transactions.
  - Load all customers with their corresponding bank balance and loan amount.
  - Find and load all senior citizen (age > 65 yrs) customers, their account balance and loan amount if any.
  - Find and load the principal amount of all loans and their corresponding interest payable for each customer.





# Summary

[www.3aayaam.in](http://www.3aayaam.in)

- Lakehouse
- Loading data into Lakehouse tables
- V-Order, Z-Order, Table Maintenance
- Spark Capacity Settings on Fabric
- Starter Pool & Custom Pool
- Environment
- NotebookUtils
- Notebook
- Spark Job Definition